

Project Report

On

Micro-Doppler Based Classification System

In partial fulfillment of requirements for the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by:

Ansh Raghuwanshi [23100BTCSE14772]

Anjali Jain [23100BTCSE14765]

Aakash Namdeo [23100BTCSE14711]

Ankit Vishwakarma [23100BTCSE14769]

Under the guidance of

Dr. Abhishek Sharma



SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE

SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING Jan

– June 2026 Session

SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE
SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

We here declare that the work which is being presented in the project entitled “**Micro-Doppler Based Classification System**” in partial fulfillment of the degree of **Bachelor of Technology in Computer Science Engineering** is an authentic record of our work carried out under the supervision and guidance of **Dr. Abhishek Sharma**. The matter embodied in this project report is original and has not been submitted earlier for the award of any degree or diploma.

Submitted By:

Ansh Raghuwanshi [23100BTCSE14772]

Anjali Jain [23100BTCSE14765]

Aakash Namdeo [23100BTCSE14711]

Ankit Vishwakarma [23100BTCSE14769]

Date: 26 May 2026

Place: Indore

SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE
SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

PROJECT APPROVAL SHEET

The following team has done the appropriate work related to the “**Micro-Doppler Based Classification System**” in partial fulfillment for the award of Bachelor of Technology in Computer Science **Engineering** and is being submitted to **SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE**.

Team:

1. Ansh Raghuwanshi
2. Anjali Jain
3. Aakash Namdeo
4. Ankit Vishwakarma

Internal Examiner

External Examiner

SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE
SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that **Mr. Ansh Raghuwanshi, Mrs. Anjali Jain, Mr. Aakash Namdeo, and Mr. Ankit Vishwakarma**, working in a team, have satisfactorily completed the project entitled “**Micro-Doppler Based Classification System**” under the guidance of **Dr. Abhishek Sharma** in the partial fulfillment of the degree of **Bachelor of Technology in Computer Science Engineering** awarded by **SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY** affiliated to **SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE** during the academic year **Jan-July 2026**.

Dr. Abhishek Sharma
Project Guide

Mr. Vijay Chauhan
Project Coordinator

Dr. Jigyasu Dubey
**Head, Department of Computer
Science Engineering
SVIIT-SVVV**

Dr. Anand Rajavat
Director, SVIIT-SVVV

ACKNOWLEDGEMENTS

We are grateful to a number of persons for their advice and support during the time of completing our project work. First and foremost, our thanks goes to **Dr. Jigyasu Dubey** Head of the Department of Computer Science and **Dr. Abhishek Sharma**, the mentor of our project for providing us valuable support and necessary help whenever required and also helping us explore new technologies by the help of their technical expertise. His direction, supervision and constructive criticism were indeed the source of inspiration for us.

We would also like to express our sincere gratitude towards our Director **Dr. Anand Rajavat** for providing us valuable support.

We are really indebted to **Mr. Vijay Chauhan**, project coordinator for helping us in each aspect of our academic's activities. We also owe our sincere thanks to all the **faculty members** of the Computer Science Department who have always been helpful.

We forward our sincere thanks to all **teaching and non-teaching staff** of the Computer Science department, SVVV Indore for providing necessary information and their kind co-operation.

We would like to thank our parents and family members, our classmates and our friends for their motivation and their valuable suggestions during the project. Last, but not the least, we thank all those people, who have helped us directly or indirectly in accomplishing this work. It has been a privilege to study at **SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE**.

PROJECT TIMELINE

Phases	Date
Phase 1: Planning & Requirement Analysis (Weeks 1–2) Focus: Defining the scope, setting up infrastructure, and identifying functional requirements. Week 1: Project Initiation - Form teams and define the problem statement. Tooling: Set up Jira boards (Scrum or Kanban), create Backlog, and initialize Git repositories. Deliverable: Project Vision document and initial Product Backlog.	19 - 24 January 2026
Week 2: Functional Modeling - Identify Actors and Use Cases. UML Deliverables: * Use Case Diagram: To define system boundaries. Use Case Descriptions: Detailed text for each primary flow. Tooling: Map Use Cases to Jira User Stories.	26 - 31 January 2026
Phase 2: Object-Oriented Analysis (Weeks 3–4) Focus: Understanding the "What" of the system without focusing on implementation. Week 3: Static Analysis - Identify domain entities and their relationships. UML Deliverables: Object Diagram: To model specific instances and data snapshots. Initial Class Diagram (Domain Model): Focus on attributes and relationships (\$1:1\$, \$1:N\$, \$M:N\$) without methods.	2 - 7 February 2026
Week 4: Dynamic Analysis - Model how the system reacts to events. UML Deliverable: Activity Diagram: To model complex business logic and parallel workflows.	9 - 14 February 2026

Phases	Date
Phase 3: Object-Oriented Design (Weeks 5–7) Focus: Defining the "How" – architecture, patterns, and component interactions. Week 5: Interaction Modeling UML Deliverable: * Sequence Diagram: Detailed logic showing how objects communicate over time to fulfill a use case.	16 - 21 February 2026
Week 6: Detailed Class Design & Packaging - Apply SOLID principles and Design Patterns. - UML Deliverables: Detailed Class Diagram: Including methods, visibility, and interfaces. Package Diagram: Organizing classes into logical modules/subsystems.	23 - 28 February 2026
Week 7: System Architecture - UML Deliverables: Component Diagram: Visualizing physical modules (API, Database, UI). Deployment Diagram: Mapping software components to hardware/nodes (Cloud, Servers).	2 - 7 March 2026
Phase 4: Implementation & CI/CD Integration (Weeks 8–10) Focus: Coding, version control, and automated pipelines. Week 8: Sprint 1 - Core Features - Develop core logic based on the Class and Sequence diagrams. Git Strategy: Use branching (Feature branches → Develop → Main).	23 - 28 March 2026
Week 9: CI/CD Pipeline Setup - Configure GitHub Actions or GitLab CI. - Automate build triggers on every git push.	30 March - 4 April 2026
Week 10: Sprint 2 - Integration - Develop UI and connect to the database. Jira: Move stories to "In Progress" and "Testing."	6 - 11 April 2026

Phases	Date
Phase 5: Testing & Deployment (Weeks 11–12) Focus: Quality assurance and final delivery. · Week 11: Testing Phase ○ Unit Testing: (JUnit, PyTest) ensuring methods work as intended. ○ Integration Testing: Testing the flow between components. ○ Tooling: Integrate test reports into the CI/CD pipeline.	13 – 18 April 2026
· Week 12: Deployment & Final Review ○ Finalize the Deployment Diagram to match the production environment. ○ Deploy to a staging/production environment (Heroku, AWS, or Azure). ○ Project Demo and handover.	20 – 25 April 2026

ABSTRACT

The increasing use of drones in modern airspace has created significant challenges in radar-based surveillance systems, particularly in distinguishing drones from birds. This project presents a Micro-Doppler Based Classification System that uses machine learning techniques to classify radar micro-Doppler feature data into Bird and Drone categories. The system utilizes a dataset containing numerical radar signal features extracted from micro-Doppler signatures. Data analysis and visualization techniques were performed to identify meaningful feature patterns suitable for classification. A supervised machine learning approach using algorithms such as Random Forest and Logistic Regression was considered for model development and evaluation. The frontend of the system was developed using React.js, while the backend was implemented using Node.js and Express. The machine learning model was developed in Python using Scikit-learn. The system was successfully designed, tested, and deployed using modern cloud platforms.

LIST OF FIGURE

FIGURES	PAGE NO.
Figure 1.1 : System Architecture Diagram	5
Figure 2.2 : Proposed System Workflow Diagram	9
Figure 4.1 : Use Case Diagram	24
Figure 4.2 : Class Diagram	25
Figure 4.3 : Activity Diagram	26
Figure 4.4 : Data flow Diagram (Level 0,1,2)	27
Figure 5.1 : Detailed Diagram	29
Figure 5.2.1 : Sequence Diagram	30
Figure 5.2.2 : Collaboration Diagram	31
Figure 5.3 : State Diagram	32
Figure 5.4 : Activity Diagram	33
Figure 5.5 : Object Diagram	34
Figure 5.6 : Deployment Diagram	35

TABLE OF CONTENTS

Declaration	I
Project Approval Sheet	II
Certificate	III
Acknowledgment	IV
Abstract	VII
List of Figure	IX
 CHAPTER 1 - INTRODUCTION	 1-6
1.1 Background	1
1.2 Problem Statement	1
1.3 Need for the Proper System	1
1.4 Objective	2
1.5 Module of the System	3
1.6 Scope	4
 CHAPTER 2 - LITERATURE SURVEY	 5-11
2.1 Existing System	7
2.2 Proposed System	8
2.3 Feasibility Study	10
2.3.1 Technical Feasibility	10
2.3.2 Economical Feasibility	10
2.3.3 Operational Feasibility	11
 CHAPTER 3 - REQUIREMENTS ANALYSIS	 12-19
3.1 Method used for Requirement analysis	12
3.2 Data Requirements	13
3.3 Functional Requirements	15
3.4 Non Functional Requirements	16
3.5 System Specification	17
3.5.1 Hardware specification	18
3.5.2 Software Specification	18
 CHAPTER 4 - DESIGN	 20-28
4.1 Software Requirements Specification	20
4.1.1 Glossary	20
4.1.2 Supplementary Specifications	22

4.1.3 Use Case Model	24
4.2 Conceptual level class diagram	25
4.3 Conceptual level activity diagram	26
4.4 Data flow Diagram (Level 0,1,2)	27
4.5 Database Design (ER-Diagram)	28
 CHAPTER 5 – SYSTEM MODELING	 29-37
5.1 Detailed Class Diagram	29
5.2 Interaction Diagram	30
5.2.1 Sequence Diagram	30
5.2.2 Collaboration Diagram	31
5.3 State Diagram	32
5.4 Activity Diagram	33
5.5 Object Diagram	34
5.6 Deployment Diagram	35
5.7 Testing	36
 CHAPTER 6 – CONCLUSION & FUTURE WORK	 38-40
6.1 Limitation of Project	38
6.2 Future Enhancement	39
 CHAPTER 7 - BIBLIOGRAPHY & REFERENCES	 41

CHAPTER - 1

INTRODUCTION

1. INTRODUCTION

1.1 Background

Radar-based target classification has become an important research area due to the rapid increase in the use of unmanned aerial vehicles (UAVs) and drones in civilian and commercial applications. One of the major challenges in radar surveillance systems is distinguishing drones from birds, as both may produce similar radar reflections. Incorrect classification may lead to false alarms and affect the reliability of monitoring systems.

Micro-Doppler signatures provide detailed information about the movement characteristics of an object, such as wing flapping in birds and rotor blade rotation in drones. These motion patterns can be analysed to identify the type of target being detected.

The Micro-Doppler-Based Classification System is designed to classify radar signal data into Bird and Drone categories using machine learning techniques. The system analyses numerical micro-Doppler feature data extracted from radar signals and applies supervised machine learning algorithms to perform classification.

The proposed system integrates a React-based frontend, a Node.js and Express backend, and a Python-based machine learning model to create a complete classification workflow. The project focuses on developing an efficient, scalable, and intelligent classification system capable of distinguishing aerial objects based on radar signal characteristics.

1.2 Problem Statement

The increasing presence of drones in monitored airspace has created significant challenges for radar surveillance systems. Traditional radar systems often face difficulty distinguishing drones from birds because both objects may generate similar radar signatures.

The problem addressed in this project is the development of a machine learning-based system capable of analysing micro-Doppler radar feature data and accurately classifying objects as either birds or drones.

The system aims to reduce false classifications by utilizing micro-Doppler motion characteristics and supervised machine learning techniques.

1.3 Need for the Proper System

A reliable classification system is necessary to improve the accuracy and effectiveness of radar-based surveillance systems. Misclassification of drones and birds may result in security concerns, unnecessary alerts, and inefficient monitoring.

The proposed system provides:

- Improved classification accuracy
- Automated analysis of radar signal data
- Faster identification of aerial objects
- Reduced dependency on manual monitoring
- Better scalability for intelligent surveillance systems

The integration of machine learning techniques allows the system to learn patterns from radar feature data and make intelligent classification decisions.

1.4 Objective

The primary objective of this project is to develop a Micro-Doppler-Based Classification System capable of distinguishing birds and drones using machine learning techniques.

The objectives of the project include:

- Analyse micro-Doppler radar feature data
- Identify useful feature patterns for classification
- Develop a supervised machine learning model
- Integrate frontend, backend, and ML components
- Generate accurate Bird or Drone predictions
- Deploy the complete system using cloud platforms

1.5 Modules of the System

The proposed system consists of the following major modules:

1. Frontend Module

Provides the user interface for entering micro-Doppler feature data and displaying classification results.

2. Backend Module

Handles API requests, communicates with the ML model, and manages data flow between system components.

3. Machine Learning Module

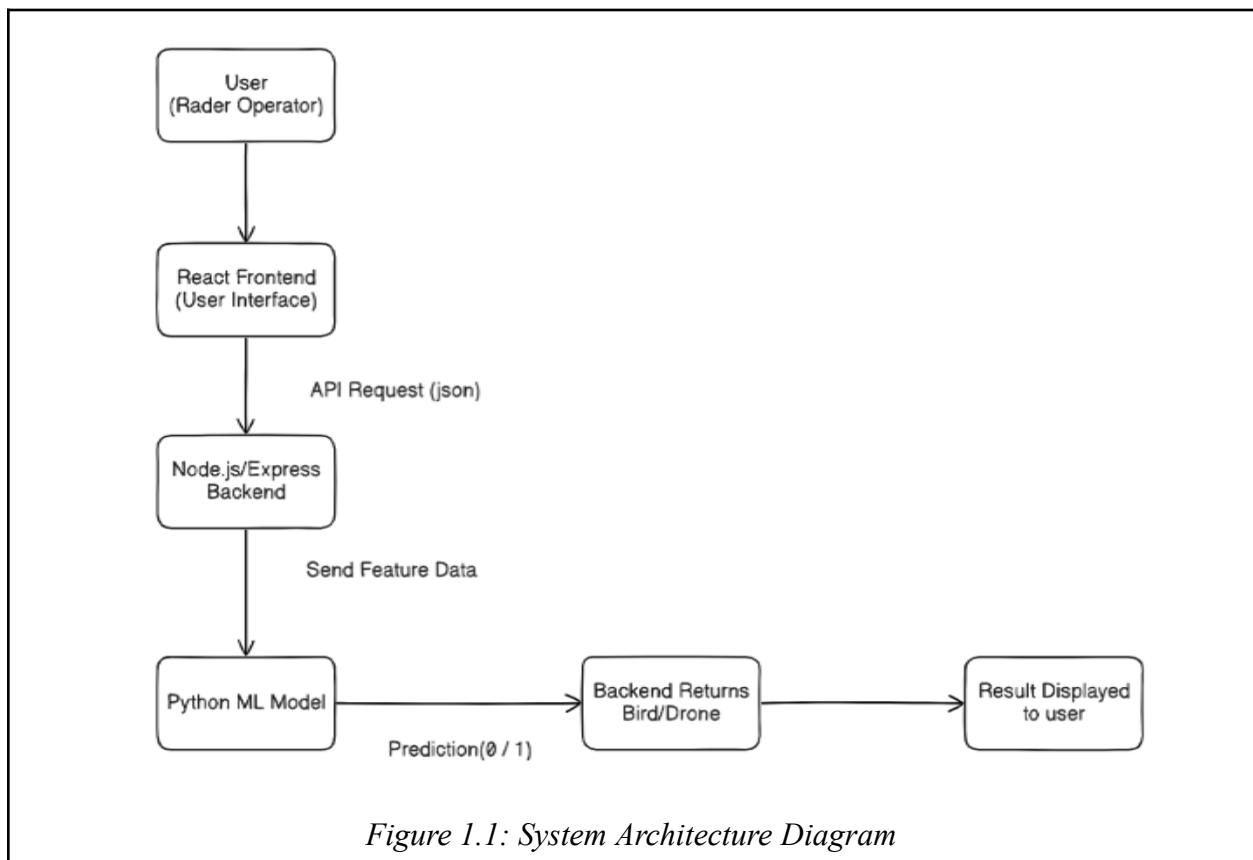
Processes radar feature data and predicts whether the input corresponds to a bird or a drone.

4. Data Processing Module

Performs data handling, preprocessing, and feature analysis on the micro-Doppler dataset.

5. Deployment Module

Manages cloud deployment and communication between deployed services.



1.6 Scope

The scope of the project includes:

- Analysis of micro-Doppler radar feature data
- Development of machine learning classification models
- Frontend and backend integration
- Deployment of the classification system

The project does not include:

- Real-time radar hardware implementation
- Live radar signal acquisition
- Military-grade deployment systems

CHAPTER - 2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Existing System

Radar-based target detection systems have traditionally been used for monitoring and surveillance applications. Conventional radar systems primarily rely on parameters such as distance, speed, and signal strength for object detection. However, distinguishing drones from birds using traditional radar methods remains a major challenge because both objects often produce similar radar reflections and movement patterns.

Existing classification systems mainly use basic signal processing techniques and conventional detection algorithms. These systems are capable of identifying the presence of moving objects but often fail to provide accurate classification between birds and drones. As a result, false alarms and incorrect detections frequently occur in surveillance systems.

Several research approaches have attempted to improve classification accuracy by using micro-Doppler signatures. Micro-Doppler analysis captures small motion characteristics such as wing flapping in birds and rotor blade rotation in drones. Although these methods improve detection performance, many existing systems still face limitations such as:

- High computational complexity
- Limited accuracy under noisy conditions
- Difficulty handling large feature datasets
- Dependence on specialized hardware systems
- Lack of scalable web-based deployment

Many existing systems are also limited to research environments and are not integrated with modern web technologies or cloud deployment platforms.

2.2 Proposed System

The proposed Micro-Doppler Based Classification System aims to overcome the limitations of traditional radar classification systems by integrating machine learning techniques with modern full-stack web technologies.

The system uses numerical micro-Doppler feature data extracted from radar signals to classify aerial objects into two categories:

- Bird
- Drone

The proposed system utilizes supervised machine learning algorithms such as Random Forest and Logistic Regression to analyse feature patterns and generate classification predictions.

Machine learning enables the system to automatically learn important patterns from the dataset and improve classification accuracy.

The architecture of the proposed system consists of:

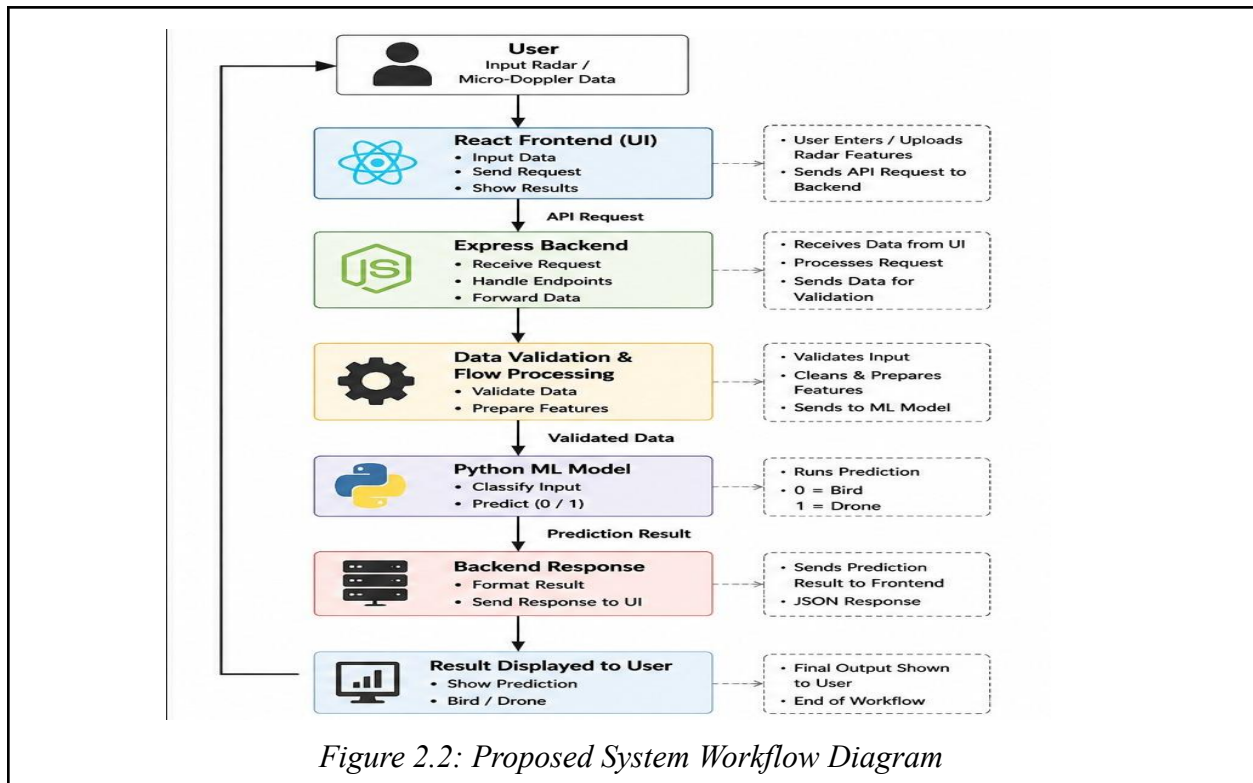
- React.js frontend for user interaction
- Node.js and Express backend for API handling
- Python-based machine learning model for prediction
- Cloud deployment using Render and Vercel

The system workflow begins when the user provides micro-Doppler feature input data through the frontend interface. The backend processes the request and forwards the feature data to the machine learning model. The model analyses the input features and predicts whether the target is a bird or a drone. The prediction result is then displayed to the user.

The proposed system provides several advantages:

- Improved classification accuracy
- Automated prediction process
- Scalable architecture
- Modern web-based interface
- Cloud deployment capability
- Better separation of frontend, backend, and ML components

The system is designed to be modular, scalable, and suitable for future enhancements.



2.3 FEASIBILITY STUDY

A feasibility study was conducted to determine whether the proposed system can be successfully developed and implemented within the available resources, technologies, and project timeline.

The feasibility analysis focuses on:

- Technical Feasibility
- Economical Feasibility
- Operational Feasibility

The study confirmed that the proposed system is feasible and can be implemented using existing technologies and machine learning frameworks.

2.3.1 TECHNICAL FEASIBILITY

The proposed system is technically feasible because all required technologies, frameworks, and tools are readily available and well-supported.

The frontend of the system is developed using React.js, which provides efficient component-based UI development. The backend is implemented using Node.js and Express.js for handling API communication and server-side processing.

The machine learning component is developed using Python and Scikit-learn, which provide efficient libraries for supervised learning and classification tasks.

The dataset required for training and testing is already available in CSV format containing numerical micro-Doppler feature data. Visualization and preprocessing can be performed using libraries such as Pandas, NumPy, and Matplotlib.

Cloud deployment platforms such as Render and Vercel provide scalable hosting solutions for the backend, frontend, and machine learning services.

The proposed architecture ensures proper communication between all components and supports modular system development.

2.3.2 ECONOMICAL FEASIBILITY

The proposed system is economically feasible because it uses open-source technologies and freely available development tools.

The project does not require expensive hardware infrastructure or specialized radar equipment during the development phase. Most development activities are performed using existing computing resources and cloud platforms with free-tier deployment support.

The technologies used in the project such as:

- React.js
- Node.js
- Python
- Scikit-learn
- GitHub
- Render
- Vercel

are open-source or freely accessible for educational use.

Therefore, the overall development cost of the project remains low while still providing a scalable and functional classification system

2.3.3 OPERATIONAL FEASIBILITY

The proposed system is operationally feasible because it provides a simple and user-friendly workflow for classification.

The frontend interface allows users to interact with the system easily by providing feature input data and receiving prediction results. The backend and machine learning components operate automatically without requiring manual intervention.

The modular architecture of the system ensures easy maintenance, future scalability, and smooth integration between frontend, backend, and machine learning services.

The system can be further extended in future for:

- Real-time radar integration
- Improved machine learning models
- Advanced visualization dashboards
- Larger datasets and improved prediction accuracy

The operational workflow confirms that the system can function effectively in practical deployment scenarios.

.

CHAPTER - 3

REQUIREMENT ANALYSIS

3. REQUIREMENT ANALYSIS

3.1 Method used for Requirement Analysis

Requirement analysis is an important phase in software development that focuses on identifying system needs, functionalities, constraints, and user expectations. For the Micro-Doppler Based Classification System, the requirement analysis phase was conducted to determine the technical and functional requirements necessary for successful implementation.

The project follows an object-oriented and modular development approach. The requirements were analyzed by studying the problem domain, understanding the workflow of radar-based classification systems, and identifying the interaction between users, backend services, and machine learning components.

The requirement analysis process included:

- Understanding the project objectives
- Identifying system actors and use cases
- Studying dataset structure and feature requirements
- Determining frontend and backend communication needs
- Defining machine learning model requirements
- Identifying deployment and integration requirements

UML diagrams such as Use Case Diagrams, Activity Diagrams, and Class Diagrams were used to model the functional behaviour and system structure.

The analysis phase helped establish a clear roadmap for system implementation and integration.

3.2 Data Requirements

The proposed system requires a dataset containing numerical micro-Doppler radar feature data for training and classification purposes.

The dataset used in the project consists of:

- Approximately 5000 numerical feature columns
- One label column representing the target class

Each row in the dataset represents a radar observation sample. The numerical features represent extracted micro-Doppler signal characteristics derived from radar reflections of moving objects.

The label column contains:

- 0 → Drone
- 1 → Bird

The dataset is stored in CSV format and is processed using Python libraries such as Pandas and NumPy.

The data requirements for the system include:

- Numerical feature consistency
- Properly labelled training data
- Sufficient feature diversity
- Structured CSV formatting
- Compatibility with machine learning algorithms

The dataset is used for:

- Data exploration
- Feature analysis
- Model training
- Model testing
- Prediction generation

3.3 Functional Requirements

Functional requirements describe the core functionalities that the system must perform.

The functional requirements of the proposed system are as follows:

1. User Input Functionality

The system must allow users to provide micro-Doppler feature input data through the frontend interface.

2. Data Processing

The system must process and validate incoming feature data before sending it to the machine learning model.

3. Classification Functionality

The system must classify the input data into:

- Bird
- Drone

using machine learning algorithms.

4. Prediction Display

The system must display the classification result clearly to the user.

5. Backend Communication

The backend must receive requests from the frontend and communicate with the Python ML model.

6. Machine Learning Processing

The ML model must analyse feature data and generate accurate predictions.

7. Error Handling

The system must handle invalid inputs and failed prediction requests appropriately.

8. Deployment Functionality

The system must support cloud-based deployment for frontend, backend, and ML services.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance expectations of the system.

The non-functional requirements include:

1. Performance

The system should generate predictions within an acceptable response time.

2. Scalability

The architecture should support future expansion and integration with larger datasets or additional services.

3. Reliability

The system should provide stable and consistent prediction results.

4. Usability

The frontend interface should be simple, user-friendly, and easy to navigate.

5. Maintainability

The modular system architecture should support easy maintenance and updates.

6. Security

API communication between frontend, backend, and ML services should be properly managed.

7. Availability

The deployed services should remain accessible and operational through cloud hosting platforms.

3.5 SYSTEM SPECIFICATION

The Micro-Doppler-Based Classification System is designed as a web-based machine learning application consisting of frontend, backend, and ML components.

The frontend interface is responsible for user interaction and data visualization. The backend server manages API communication and system workflow. The Python-based machine learning model performs feature analysis and classification.

The system architecture supports:

- REST API communication
- Machine learning prediction services

- Cloud deployment
- Modular software design

The classification workflow involves:

1. Receiving feature input data
2. Processing and validating data
3. Performing ML-based classification
4. Returning prediction results
5. Displaying output to the user

3.5.1 HARDWARE SPECIFICATION

The hardware requirements for the development and deployment of the system are minimal because the project primarily uses cloud-based services and lightweight machine learning frameworks.

Minimum Hardware Requirements

- Processor: Intel i5 or higher
- RAM: 8 GB minimum
- Storage: 256 GB SSD
- Internet Connectivity
- Standard Keyboard and Display

The system does not require specialized radar hardware during development and testing.

3.5.2 SOFTWARE SPECIFICATION

The following software technologies and tools were used for system development:

Frontend Technologies

- React.js
- HTML
- CSS
- JavaScript

Backend Technologies

- Node.js
- Express.js

Machine Learning Technologies

- Python
- Scikit-learn
- Pandas
- NumPy
- Matplotlib

Development Tools

- Google Colab
- VS Code
- Git
- GitHub

Deployment Platforms

- Render
- Vercel

CHAPTER - 4

DESIGN

4. DESIGN

4.1 Software Requirements Specification

4.1.1 Glossary

Term	Description
Micro-Doppler	Small frequency variations caused by motion characteristics such as wing flapping or rotor movement
Classification	Process of categorizing input data into predefined classes
ML Model	Machine Learning model used for prediction
Feature Data	Numerical values extracted from radar signals
API	Application Programming Interface used for communication between the frontend and backend
Frontend	User interface of the application
Backend	Server-side logic handling requests and communication
Random Forest	Supervised machine learning algorithm used for classification

4.1.2 Supplementary Specification

The supplementary specifications describe additional quality attributes and operational requirements of the system.

Performance Requirements

- The system should provide prediction results within acceptable response time.
- Backend API communication should remain stable under normal usage conditions.

Security Requirements

- API requests should be properly validated.
- Invalid data inputs should be handled securely.

Scalability Requirements

- The architecture should support future integration of larger datasets and additional ML models.

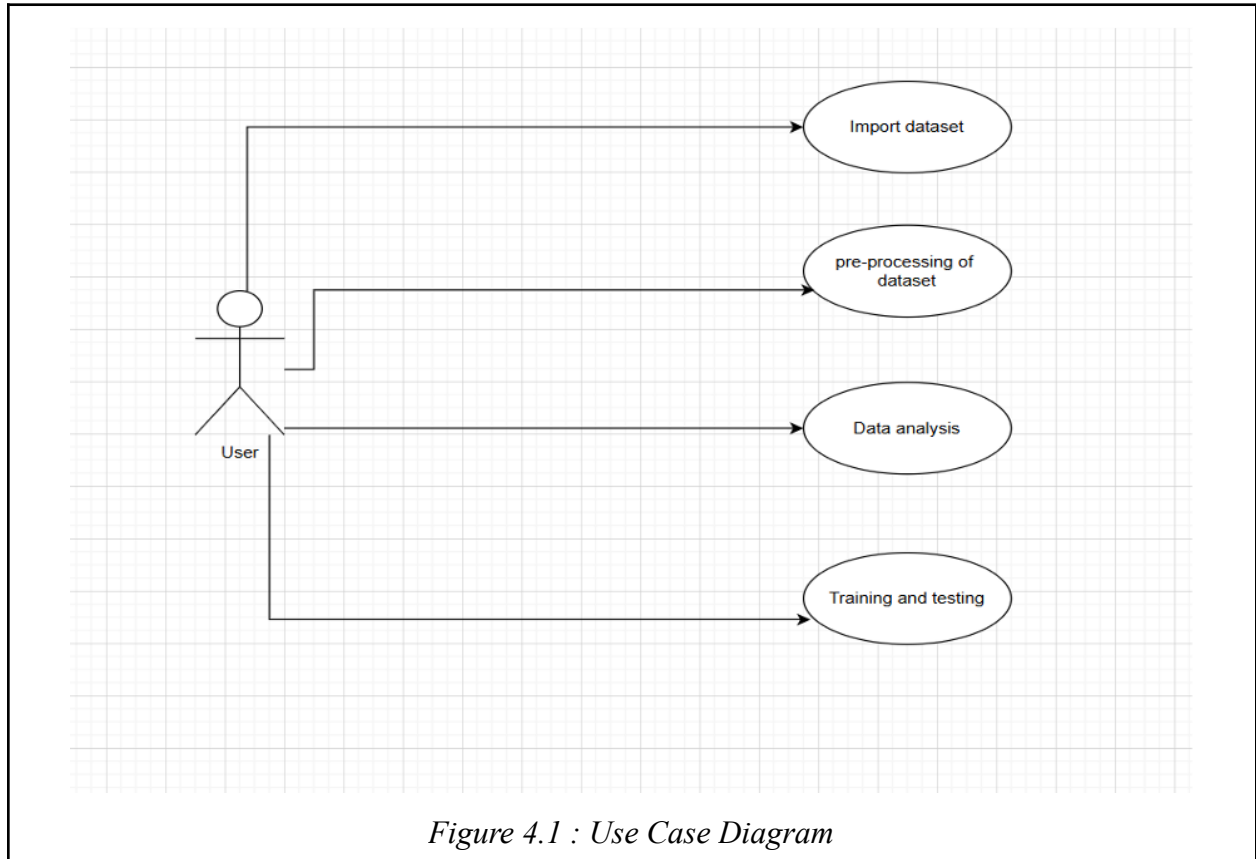
Reliability Requirements

- The system should provide consistent prediction results for valid inputs.

Maintainability Requirements

- The modular architecture should support easy maintenance and future upgrades.

4.1.3 Use Case Model



4.2 Conceptual Level Class Diagram

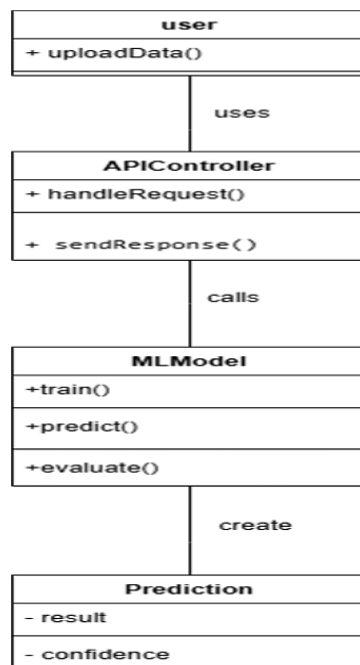


Figure 4.2 : Class Diagram

4.3 Conceptual level activity diagram

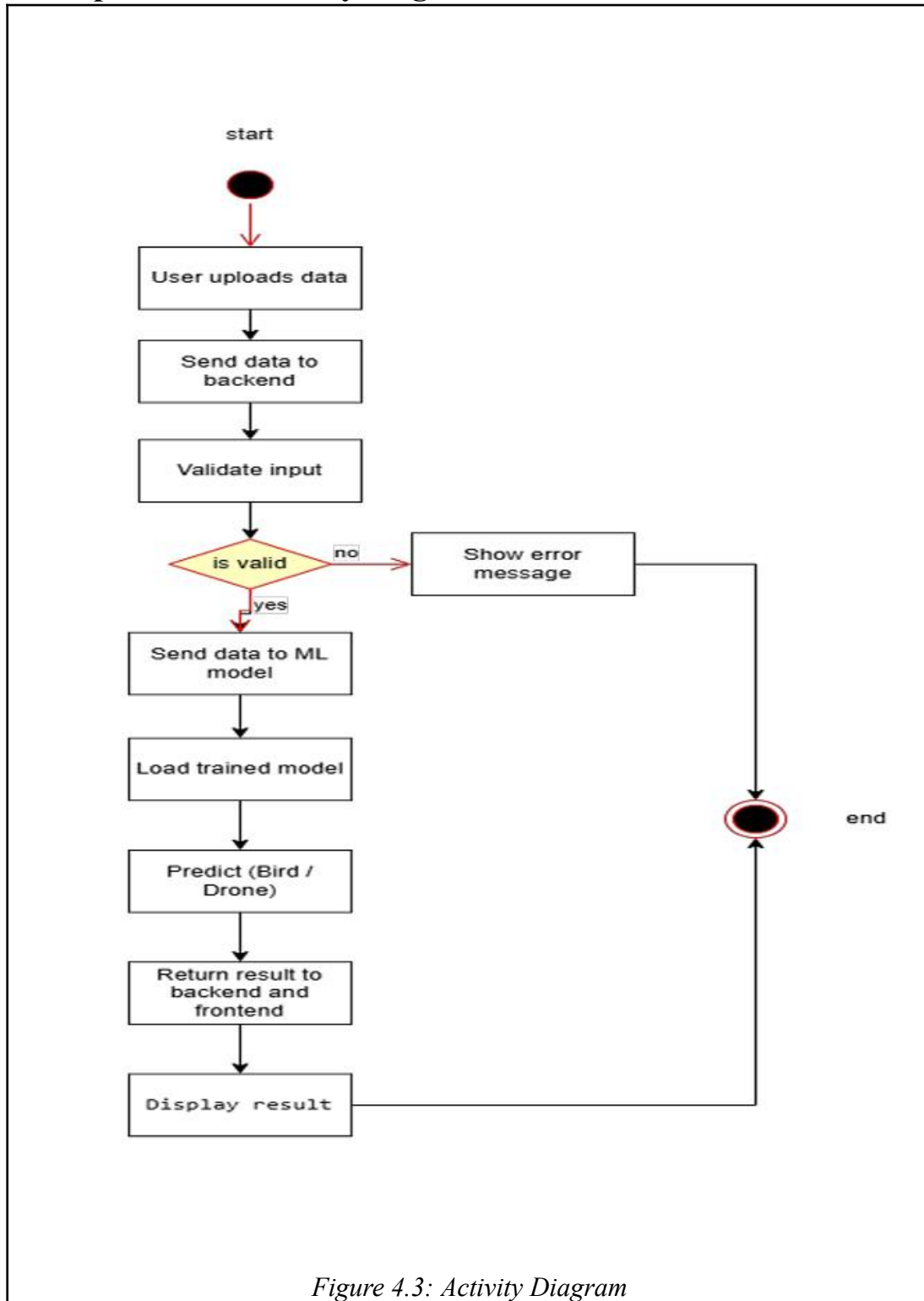


Figure 4.3: Activity Diagram

4.4 Data flow Diagram (Level 0,1,2)

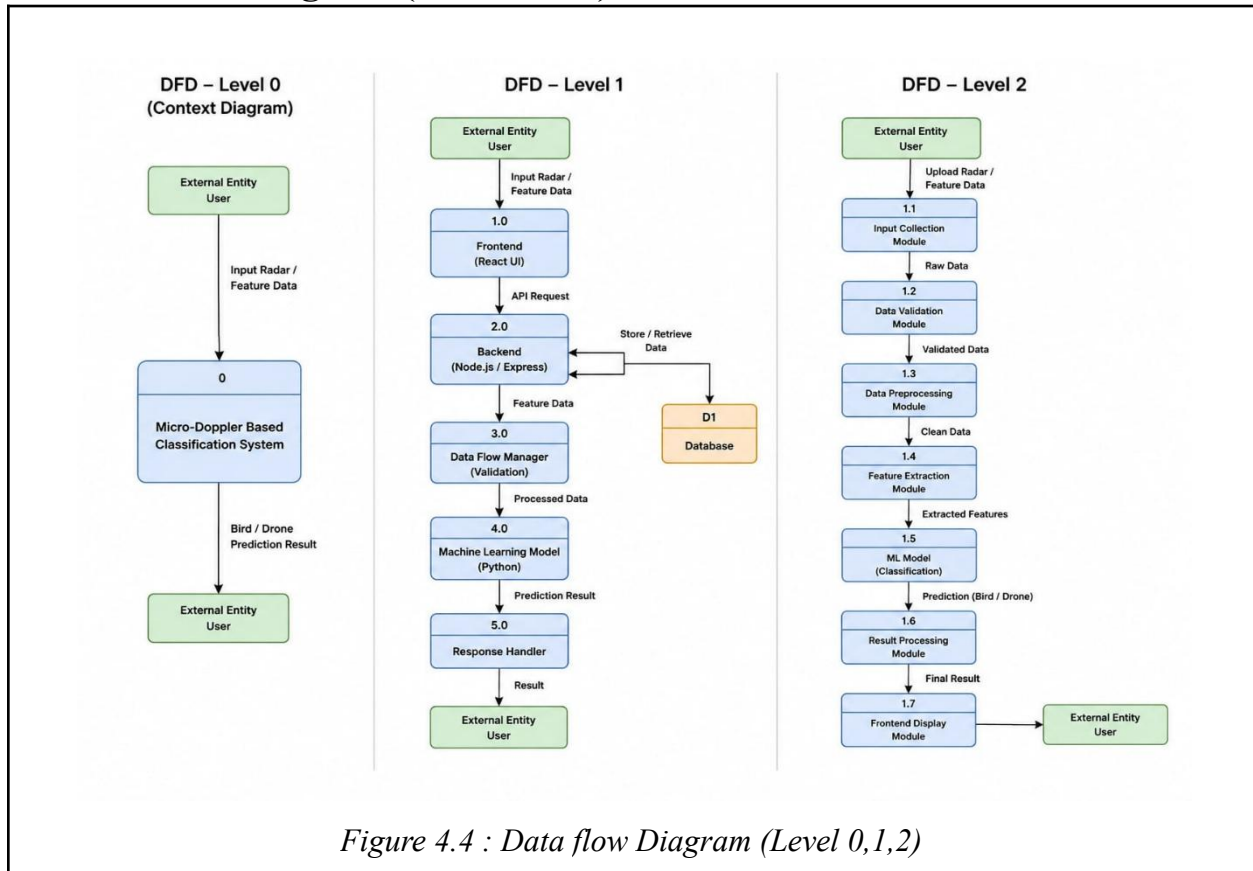
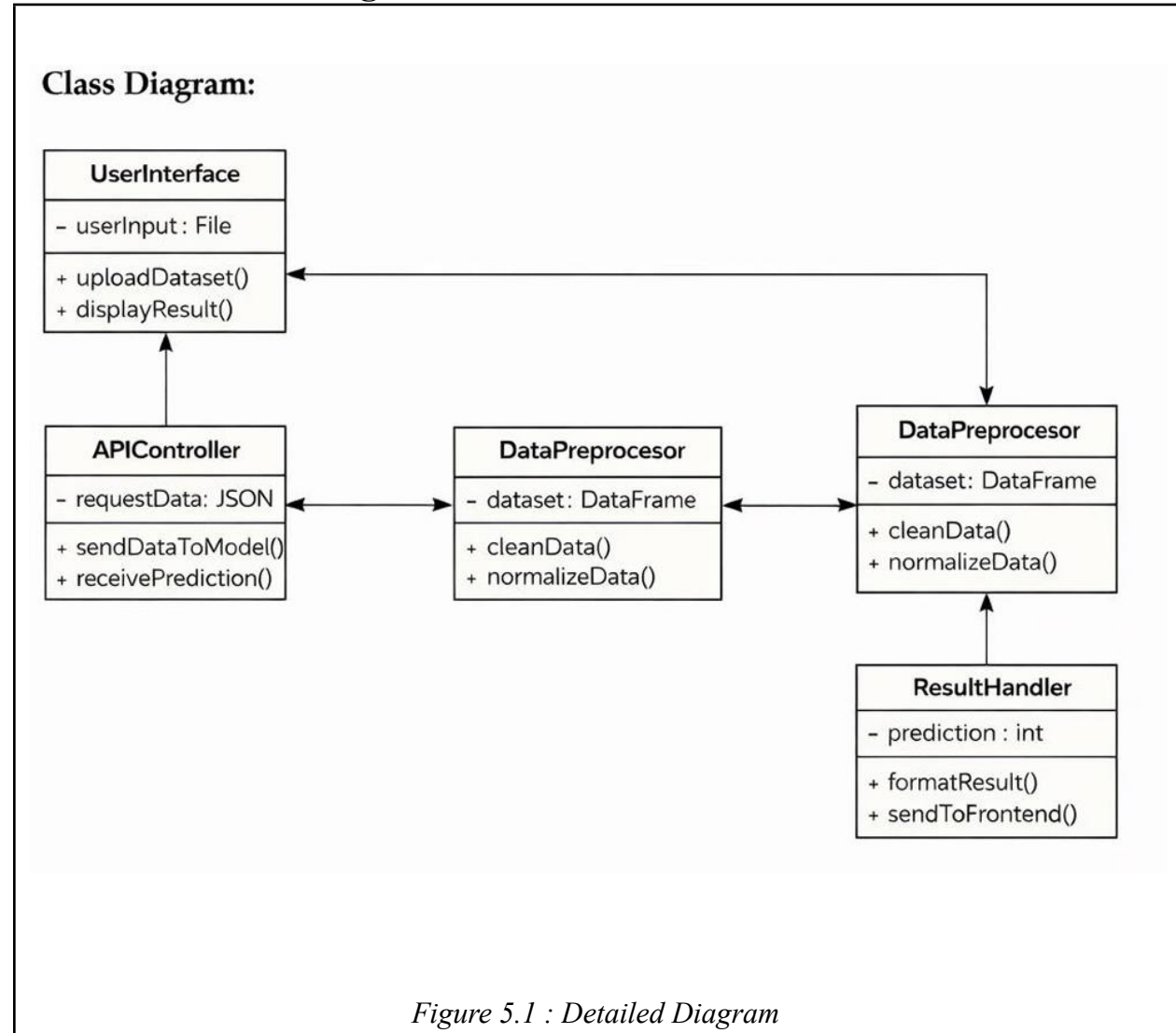


Figure 4.4 : Data flow Diagram (Level 0,1,2)

5. SYSTEM MODELING

5.1 Detailed Class Diagram



5.2 Interaction Diagram

5.2.1 Sequence Diagram

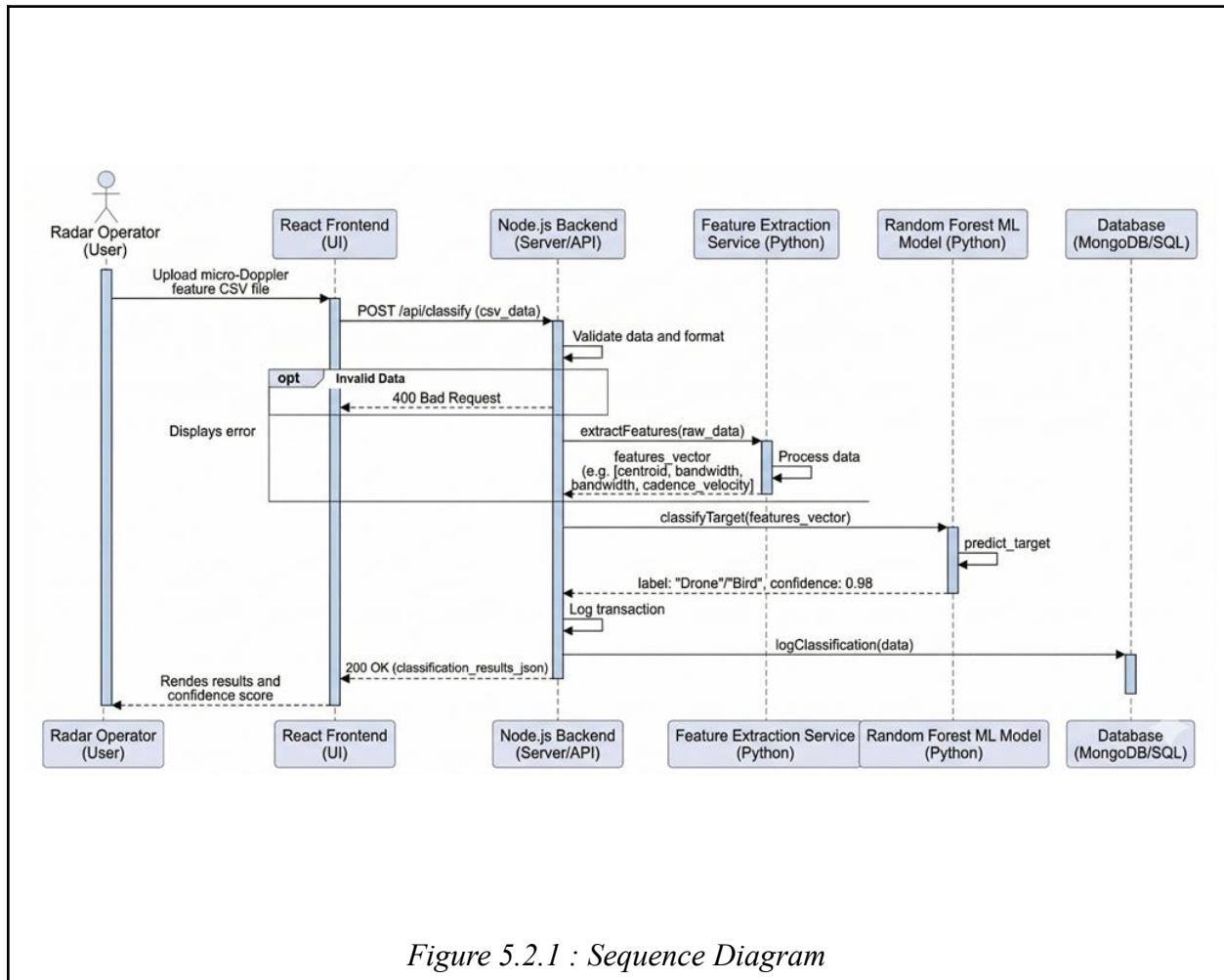


Figure 5.2.1 : Sequence Diagram

5.2.2 Collaboration Diagram

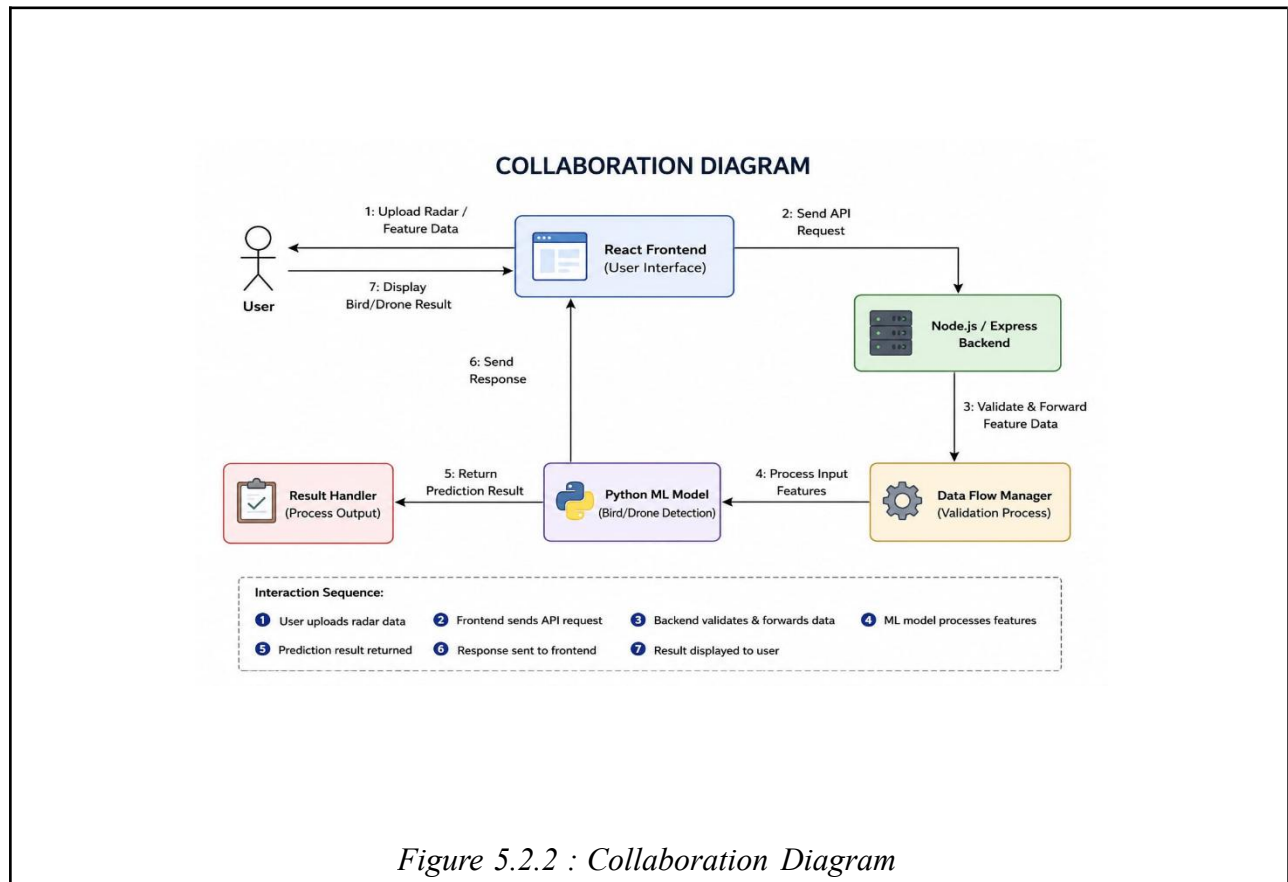


Figure 5.2.2 : Collaboration Diagram

5.3 State Diagram

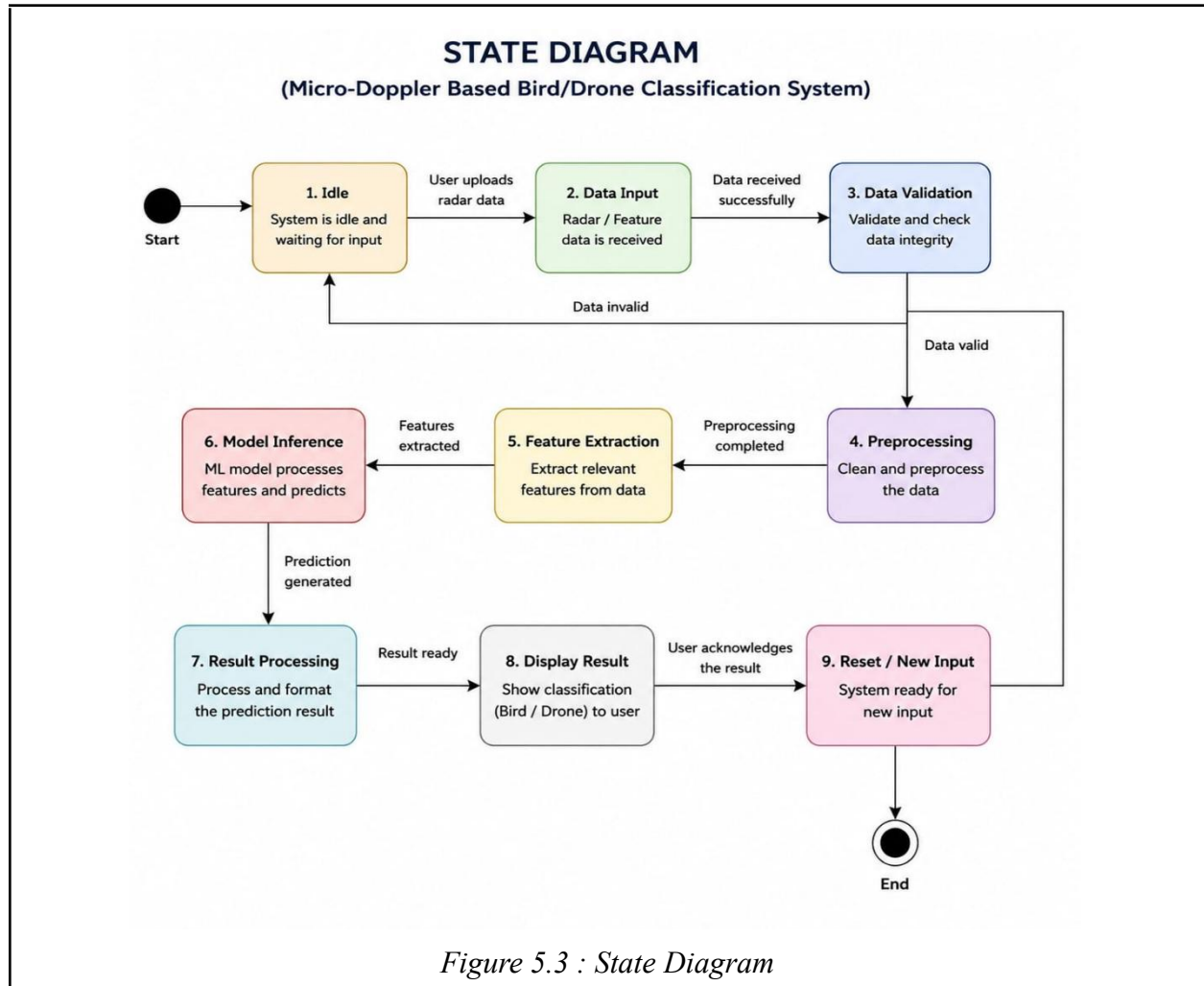
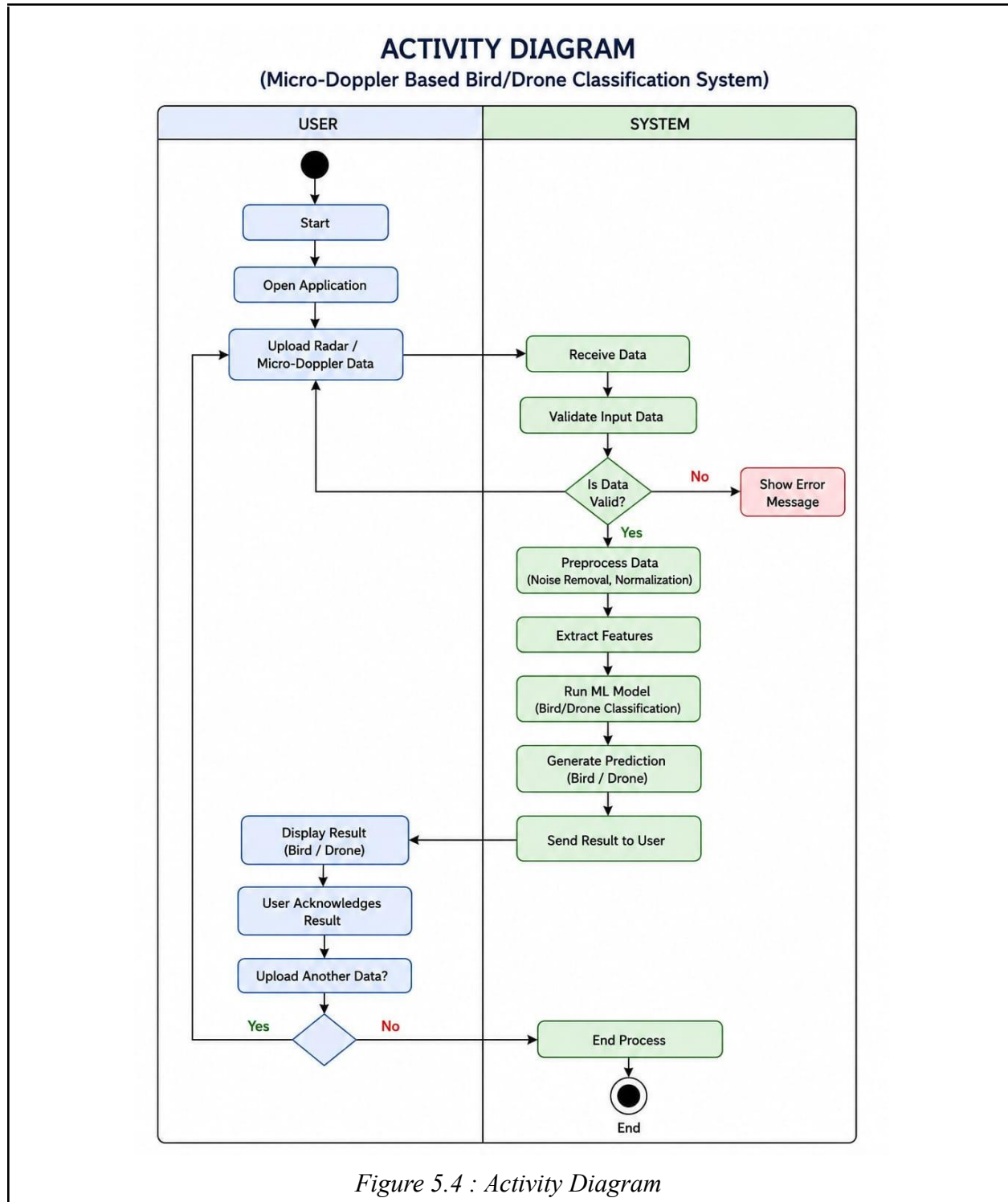


Figure 5.3 : State Diagram

5.4 Activity Diagram



5.5 Object Diagram

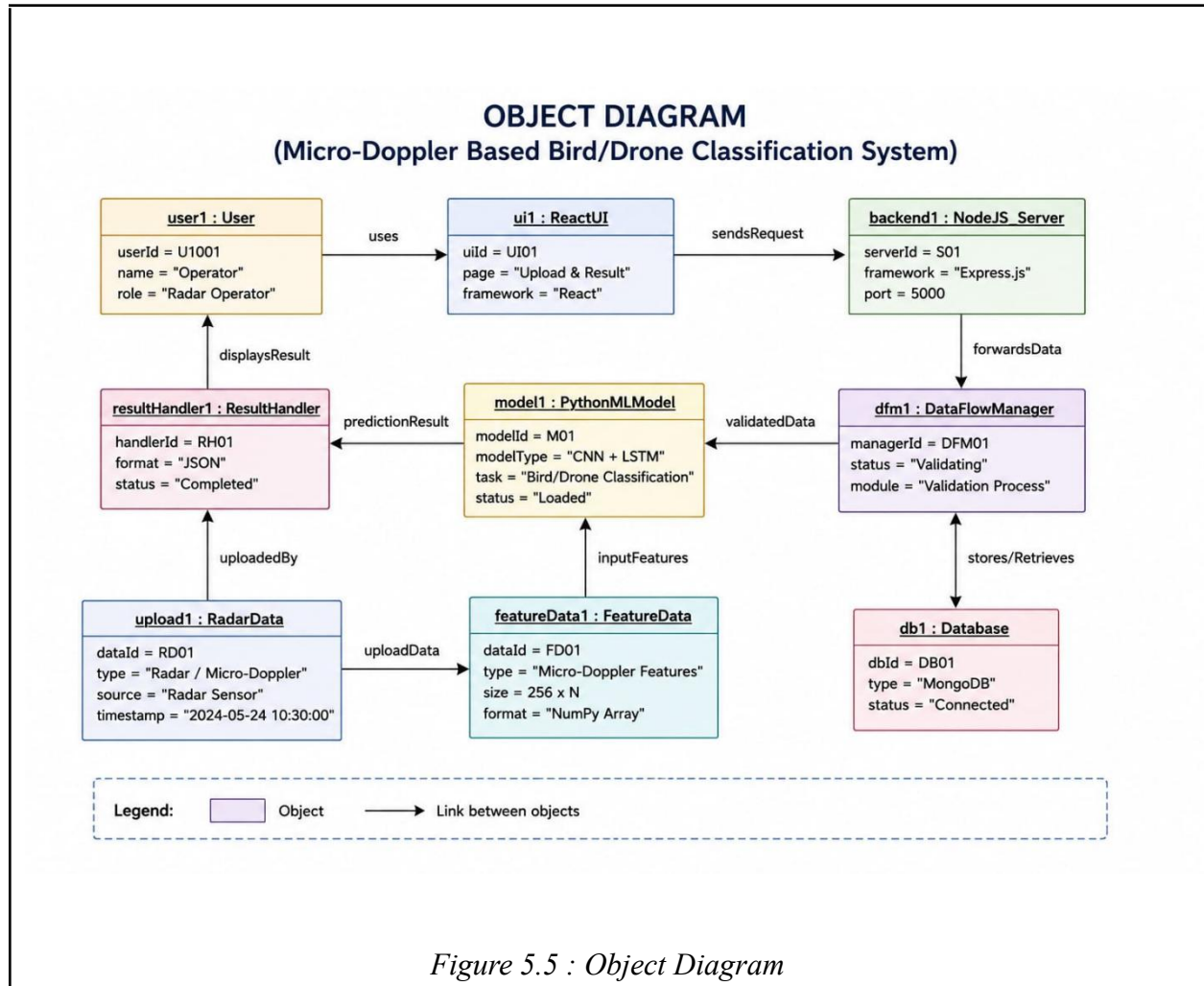


Figure 5.5 : Object Diagram

5.6 Deployment Diagram

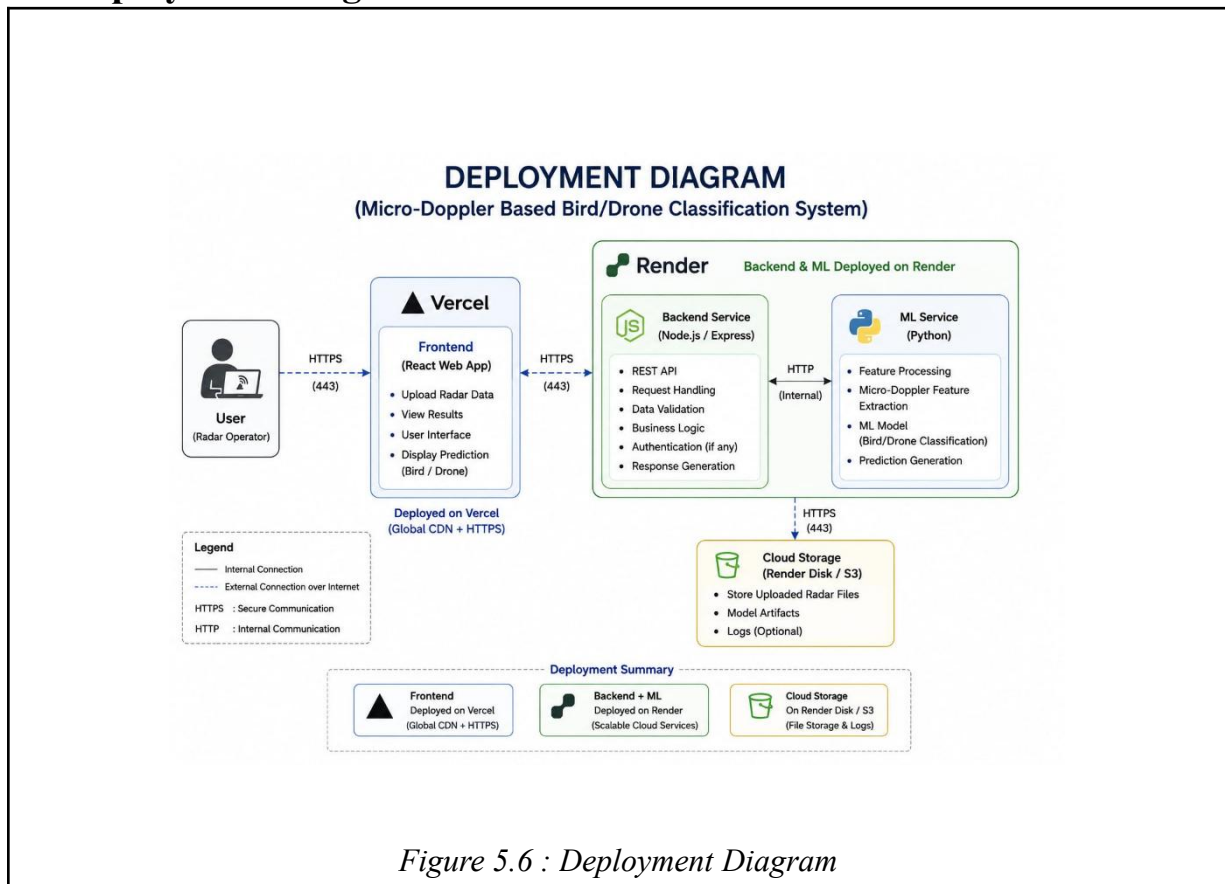


Figure 5.6 : Deployment Diagram

5.7 Testing

Testing is an important phase of software development used to verify that the functions work correctly according to the specified requirements. The system was tested using different testing methods to ensure reliability, security, accuracy, and proper performance.

The following types of testing were performed in the system:

1. Unit Testing

- Unit testing was performed to verify the functionality of individual modules independently.
- Each module was tested separately to ensure proper execution without errors.

Frontend Testing

- Input validation
- Button functionality
- Result rendering
- User interaction handling

Backend Testing

- API endpoint testing
- Request handling
- Response generation
- Error handling mechanisms

ML Model Testing

- Prediction generation
- Feature processing
- Invalid input handling
- Classification result verification

2. Integration Testing

- Integration testing was performed to ensure smooth communication between different system modules.

Modules Tested

- Frontend and Backend
- Backend and ML Model

The Testing Verified

- Proper API communication
 - JSON data transfer
 - Successful prediction workflow
 - Smooth data flow between components
- The testing ensured that all integrated modules worked together efficiently without communication failures.

3. System Testing

Complete system testing was performed to evaluate the end-to-end workflow of the application.

The following operations were validated:

- User input processing
 - API communication
 - ML prediction generation
 - Result display
 - Error handling
 - Overall system performance
- System testing verified that the application successfully completed the full prediction workflow from input processing to final result generation.

CHAPTER - 6

CONCLUSION & FUTURE WORK

6.1 Limitations of the Project

Although the Micro-Doppler-Based Bird and Drone Classification System provides an efficient solution for object classification using machine learning techniques, the current system still has certain limitations.

The major limitations of the project are as follows:

1. Dependency on Input Data Quality

The accuracy of the system highly depends on the quality of the radar or micro-doppler input data. Noisy or incomplete signal data may reduce prediction accuracy.

2. Limited Dataset Availability

The current model is trained using a limited dataset for bird and drone classification. Larger and more diverse datasets are required to improve generalization and prediction performance.

3. Internet Dependency

The web application requires a stable internet connection for accessing the frontend, backend services, and prediction modules.

4. Limited Real-Time Processing

The current implementation focuses mainly on classification accuracy and basic prediction flow. Real-time high-speed signal processing is limited.

5. Hardware Constraints

The system performance may vary depending on the system hardware specifications. Low-performance systems may experience slower prediction processing.

6. Limited Automation

Although the system automates classification tasks, advanced automation, such as automatic radar signal extraction and continuous monitoring, is not fully implemented.

7. Scalability Constraints

The current architecture is designed for academic and small-scale deployment. Large-scale deployment for defence or surveillance applications may require cloud infrastructure and distributed processing.

8. No Offline Functionality

The system is completely web-based and does not support offline prediction or local standalone deployment.

9. Security Challenges

Basic authentication and API security mechanisms are implemented; however, advanced protection against cyber threats, unauthorized access, and secure communication requires further enhancement.

10. Limited Classification Categories

The current model primarily focuses on Bird vs. Drone classification. It does not support the classification of multiple aerial object categories.

6.2 Future Enhancement

The Micro-Doppler-Based Bird and Drone Classification System can be further improved by integrating advanced technologies and additional functionalities to enhance system performance, scalability, security, and prediction accuracy.

The following future enhancements can be implemented in the system:

1. Real-Time Radar Signal Processing

Future versions can implement real-time radar signal acquisition and processing for live object classification.

2. Advanced Deep Learning Models

Advanced Deep Learning techniques such as CNN, RNN, and Transformer-based models can be integrated to improve classification accuracy.

3. Large-Scale Dataset Integration

The system can be trained using larger and more diverse radar datasets to improve robustness and model generalization.

4. Cloud Deployment

The application can be deployed on cloud platforms such as AWS, Microsoft Azure, or Google Cloud to improve scalability, performance, and availability.

5. Multi-Object Classification

Future systems can support the classification of multiple aerial objects such as helicopters, airplanes, UAVs, and birds.

6. Edge Device Integration

The model can be optimized for deployment on embedded and edge devices for low-latency prediction and portable surveillance systems.

7. Automated Signal Extraction

Automation techniques can be integrated to automatically capture and preprocess radar signals without manual intervention.

8. Enhanced Security Mechanisms

Advanced security features such as encrypted API communication, secure authentication, and intrusion detection systems can be implemented.

9. Mobile Application Support

Dedicated Android and iOS applications can be developed to allow remote monitoring and prediction access.

10. AI-Based Monitoring System

Artificial Intelligence can be integrated for intelligent threat detection, anomaly identification, and automated surveillance monitoring.

11. Integration with IoT Systems

The system can be connected with IoT-enabled radar and surveillance devices for smart monitoring applications.

CHAPTER - 7

BIBLIOGRAPHY & REFERENCES

7. BIBLIOGRAPHY & REFERENCES

1. React Official Documentation: <https://react.dev>
2. Node.js Official Documentation: <https://nodejs.org>
3. Express.js Official Documentation: <https://expressjs.com>
4. Python Official Documentation: <https://www.python.org>
5. Scikit-learn Documentation: <https://scikit-learn.org>
6. IEEE Xplore Digital Library: <https://ieeexplore.ieee.org>